Checkout Service

Regression & Incident Test Cases

Incident: 2026-06-09 · Deployment v2.7.0 · Engineering QA

Test Case Index

TC-001

Cache Key Mismatch Causes Cart Retrieval Miss

COMPONENT

CartService / CacheService

PRIORITY / TYPE / STATUS

P0 – Critical · Regression · FAIL

DESCRIPTION

After deployment v2.7.0, CartRepository reads the cache using the prefixed key 'cart_<cartId>' while CartWriter still writes with the bare 'cartId'. This asymmetry guarantees every read is a cache miss, forcing a DB fallback and ultimately causing checkout failures.

PRE-CONDITIONS

v2.7.0 deployed at 09:00 UTC on 2026-06-09. A cart for userId=18271 exists in the database. CacheService is running normally.

TEST STEPS

1. User initiates a checkout (userId=18271).
   → *CheckoutService receives request.*
2. CartRepository.get(cartId) queries cache with key 'cart_CART-18271'.
   → *Cache miss — key was stored as 'CART-18271'.*
3. System falls back to database lookup.
   → *WARN: Cart not found for userId=18271 is logged.*
4. CheckoutService attempts to validate the null cart.
   → *ERROR: NullPointerException at CheckoutProcessor.java:88.*
5. API Gateway returns the response.
   → *POST /checkout returns HTTP 500.*

EXPECTED RESULT

Cart retrieved from cache; checkout completes with HTTP 200.

ACTUAL RESULT

Cache miss every time; checkout fails with NullPointerException and HTTP 500.

LOG EVIDENCE

```
09:07:13 WARN CartService Cart not found for userId=18271
09:08:21 WARN CacheService Cache miss for cartId=CART-18271
09:11:44 ERROR CheckoutService NullPointerException at CheckoutProcessor.java:88
```

CODE DIFF REFERENCE

```
CartRepository.java - return cache.get(cartId);
+ return cache.get("cart_" + cartId);
CartWriter.java - cache.set(cartId, cart); // write key unchanged
```

TC-002

Cart Write–Read Round-Trip Key Consistency

COMPONENT

CartWriter / CartRepository

PRIORITY / TYPE / STATUS

P0 – Critical · Unit / Integration · FAIL

DESCRIPTION

Validates that a cart written to the cache by CartWriter can be retrieved by CartRepository using the exact same key. Both classes must use the 'cart_<cartId>' prefix so write and read keys match.

PRE-CONDITIONS

CacheService is running. A test cart object is available. Both CartWriter and CartRepository point to the same cache instance.

TEST STEPS

1. Call CartWriter.write(cartId='CART-99', cart=testCart).
   → *Cart stored in cache.*
2. Immediately call CartRepository.get('CART-99').
   → *Cache is queried.*
3. Assert the returned object equals testCart.
   → *Objects should be equal with no DB fallback triggered.*
4. Check the cache hit counter metric.
   → *Hit counter incremented by 1.*

EXPECTED RESULT

CartRepository returns testCart from cache on first read with zero DB calls.

ACTUAL RESULT

CartRepository returns null; DB fallback triggered due to key prefix mismatch.

LOG EVIDENCE

```
09:08:21 WARN CacheService Cache miss for cartId=CART-18271
09:08:22 INFO Database Loading cart from database
```

CODE DIFF REFERENCE

```
Fix: CartWriter.java + cache.set("cart_" + cartId, cart);
CartRepository.java + return cache.get("cart_" + cartId);
```

TC-003

Checkout Failure Rate Threshold Alerting

COMPONENT

Metrics / Alerting

PRIORITY / TYPE / STATUS

P1 – High · Observability · FAIL

DESCRIPTION

Verifies that the monitoring system raises an alert once the checkout failure rate breaches the 5% SLA threshold. The alert should fire by 09:12 UTC so on-call engineers are paged promptly after the bad deployment.

PRE-CONDITIONS

v2.7.0 is deployed and producing checkout failures. Metrics pipeline is operational. Alert threshold is configured at 5% failure rate.

TEST STEPS

1. Deploy v2.7.0 and allow checkout traffic to flow normally.
   → *Checkout failures begin accumulating from 09:08.*
2. Monitor the Metrics failure-rate gauge every minute.
   → *Rate climbs; Metrics logs 12% at 09:10.*
3. Verify an alert fires when rate exceeds 5%.
   → *Alert notification sent to on-call channel.*
4. Confirm alert timestamp is at or before 09:12 UTC.
   → *Timeline milestone 'Error rate exceeded 5%' at 09:12 should match.*

EXPECTED RESULT

Alert fires at or before 09:12 UTC when failure rate crosses 5%.

ACTUAL RESULT

Metrics logged 12% at 09:10; investigation only started at 09:18 — 6-minute gap suggests alert-to-page delay.

LOG EVIDENCE

```
09:10:01 WARN Metrics Checkout failure rate increased to 12%
Timeline: 09:12 – Error rate exceeded 5%
Timeline: 09:18 – Engineering investigation started
```

TC-004

Rollback Restores Checkout Success Rate

COMPONENT

CheckoutService / Deployment

PRIORITY / TYPE / STATUS

P1 – High · Rollback / Recovery · PASS

DESCRIPTION

Confirms that rolling back from v2.7.0 to the previous stable release fully restores the checkout success rate to its pre-incident baseline within a reasonable recovery window.

PRE-CONDITIONS

v2.7.0 is running and producing failures. Previous stable artifact is available in the deployment registry. Rollback runbook is accessible to on-call engineers.

TEST STEPS

1. Trigger rollback at 09:31 UTC.
   → *Deployment pipeline begins reverting to prior version.*
2. Wait for rollback completion signal.
   → *Rollback completed at 09:38 UTC.*
3. Monitor checkout success rate for the next 5 minutes.
   → *Rate climbs back toward baseline.*
4. Confirm success rate returns to normal by 09:42 UTC.
   → *Timeline milestone confirmed.*

EXPECTED RESULT

Checkout success rate returns to pre-incident baseline within 10 minutes of rollback.

ACTUAL RESULT

Success rate normalised at 09:42, 4 minutes after rollback completed at 09:38. Recovery confirmed.

LOG EVIDENCE

```
Timeline: 09:31 – Rollback initiated
Timeline: 09:38 – Rollback completed
Timeline: 09:42 – Checkout success rate returned to normal
```

TC-005

Cart Validation Rejects Null Cart Gracefully

COMPONENT

CheckoutService / CartValidator

PRIORITY / TYPE / STATUS

P1 – High · Error Handling · FAIL

DESCRIPTION

When the cart object is null due to a cache miss combined with a DB miss, CheckoutProcessor should throw a structured CartNotFoundException rather than an unhandled NullPointerException. The incident shows a raw NPE propagated to the API Gateway, returning an opaque HTTP 500.

PRE-CONDITIONS

CartRepository returns null for the given cartId. No cart record exists in the database for the user. CheckoutService v2.7.0 is under test.

TEST STEPS

1. Submit a checkout request for a userId whose cart is null in cache and DB.
   → *CartRepository.get() returns null.*
2. CheckoutProcessor.process(null) is invoked.
   → *Should detect null and throw CartNotFoundException.*
3. Observe the exception type propagated to ApiGateway.
   → *Should be CartNotFoundException, not NullPointerException.*
4. Verify the HTTP response code returned to the client.
   → *Should return HTTP 422 with an actionable error message.*

EXPECTED RESULT

CartNotFoundException thrown; API returns HTTP 422 with actionable error message.

ACTUAL RESULT

NullPointerException at CheckoutProcessor.java:88; API Gateway returns HTTP 500 with no client-facing detail.

LOG EVIDENCE

```
09:09:11 ERROR CheckoutService Cart validation failed
09:09:13 ERROR ApiGateway POST /checkout 500
09:11:44 ERROR CheckoutService NullPointerException at CheckoutProcessor.java:88
```
